method to call, *while* the code is running. This technique is called *Dynamic Dispatch*, and you'll find it wildly useful. To help reveal its magic, let's look at a couple of real-life examples.

Spell: Dynamic Dispatch

Method Names and Symbols

People who are new to the language are sometimes confused by Ruby's symbols. Symbols and strings belong to two separate and unrelated classes:

```
:x.class  # => Symbol
"x".class # => String
```

Nevertheless, symbols are similar enough to strings that you might wonder what's the point of having symbols at all. Can't you just use regular strings everywhere?

There are a few different reasons to use symbols in place of regular strings, but in the end the choice boils down to conventions. In most cases, symbols are used as names of things—in particular, names of metaprogramming-related things such as methods. Symbols are a good fit for such names because they are *immutable*: you can change the characters inside a string, but you can't do that for symbols. You wouldn't expect the name of a method to change, so it makes sense to use a symbol when you refer to a method name.

For example, when you call `Object#send`, you need to pass it the name of a method as a first argument. Although `send` accepts this name as either a symbol or a string, symbols are usually considered more kosher:

```
# rather than: 1.send("+", 2)
1.send(:+, 2)   # => 3
```

Regardless, you can easily convert from string to symbol and back:

```
"abc".to_sym    #=> :abc
:abc.to_s       #=> "abc"
```

The Pry Example

One example of Dynamic Dispatch comes from Pry. Pry is a popular alternative to irb, Ruby's command-line interpreter. A `Pry` object stores the interpreter's configuration into its own attributes, such as `memory_size` and `quiet`:

methods/pry_example.rb

```
require "pry"

pry = Pry.new
pry.memory_size = 101
pry.memory_size        # => 101
pry.quiet = true
```